Freight Rate Prediction

Data, validation, model and results

Hasin Ahmed

October 2026

Contents

1 Summary **2**

2 Problem and data **2**

3 Data quality and preprocessing **3**

4 Validation design **4**

5 Feature engineering **5**

6 Improvement journey **6**

7 Tuning **7**

8 Final evaluation **8**

9 December forecast **10**

10 Limitations **11**

11 Reproducibility **11**

A Final model parameters **13**

B MLflow run inventory **14**

1 Summary

The task. We predict the posted rate, in dollars, for 12,000 truckload shipments ("loads") moving in November and December 2025. We learn from 48,000 labelled loads from January to October 2025. We also draw a daily December forecast for one fixed lane.

Why it is hard. The scoring months come after every training month, so this is a forecast, not an interpolation. About 1.41% of the training rates are recording errors. 12.06% of the scoring loads touch cities that never appear in training.

The model. A two-stage model. Stage one is a ridge regression[1] on the logarithm of the rate. It describes the load (distance, weight, location, equipment) and the market (the daily market index, a slow upward drift and a price ramp near quarter ends). Stage two is a small LightGBM tree ensemble [4] that corrects what stage one leaves over. Recent loads count more than old ones in training.

Headline numbers. We measure error as the mean absolute error in dollars on loads with plausible labels, re-weighted to the scoring mix of new and known cities ("wMAE", Section 4).

- Development folds: \$42.30 (1.78% of the rate), against \$157.55 for the simplest sensible price and \$111.25 for the original CatBoost submission.
- Locked test (September–October 2025, evaluated once): \$38.14 (1.56%), against \$64.31 for the original submission.
- December 31 chart value: \$877.62.

Main caveats. Tuning bought \$0.36 on the development folds, but on the locked test the untuned version was slightly better; the two are practically equivalent. The test period had already been inspected by earlier work, so its numbers are mildly optimistic. December relies on the quarter-end ramp recurring.

2 Problem and data

Files. Four CSV files define the task.

- `train-test.csv`: 48,000 loads from 1 January to 31 October 2025, with the posted rate.
- `validation.csv`: 12,000 loads from 1 November to 31 December 2025, without rates. These are the loads we must price.
- `validation-predictions-template.csv`: the load IDs, in the order the submission must follow.
- `december-chart-inputs.csv`: one load (Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb) for every day of December.

Columns. Each load has a pickup and delivery city with coordinates, the road distance, the equipment type (Dry Van, Reefer or Flatbed), the weight, the date, a `market_index` and a `quote_signal`.

Target. The posted rate in dollars. We model its logarithm, so errors are relative and a 2% miss counts the same on a \$300 and a \$5,000 load. The forecast is the exponential of the predicted logarithm, which estimates the median rate.

[1] Linear regression with a small penalty on large coefficients, which keeps estimates stable [2].

Horizon. On 31 October we know every label so far and must price loads 1 to 61 days ahead. The market index of each target day is known, because it is a column of the scoring file.

Unseen cities. 8 cities in the scoring file never appear in training (Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego). They touch 12.06% of the scoring loads. A model that memorises city names cannot price them, so the model uses coordinates instead.

Metric. The assessment does not name a scoring metric. We chose the mean absolute error in dollars (Section 4) and report several others next to it.

3 Data quality and preprocessing

All fixes in this section use only the load's own columns or other loads on the same date. None reads the rate, so they can run before any split without leaking information.

Corrupted labels. For each load we compare its rate per mile with the median of its lane and equipment. Clean loads sit in a narrow peak. 677 loads (1.41%) sit in two separate bands, at about 0.2–0.5 and 2–5 times the lane norm, with almost nothing in between (Figure 1). They appear at the same rate in every month and equipment type, and no feature predicts them. We treat them as recording errors.

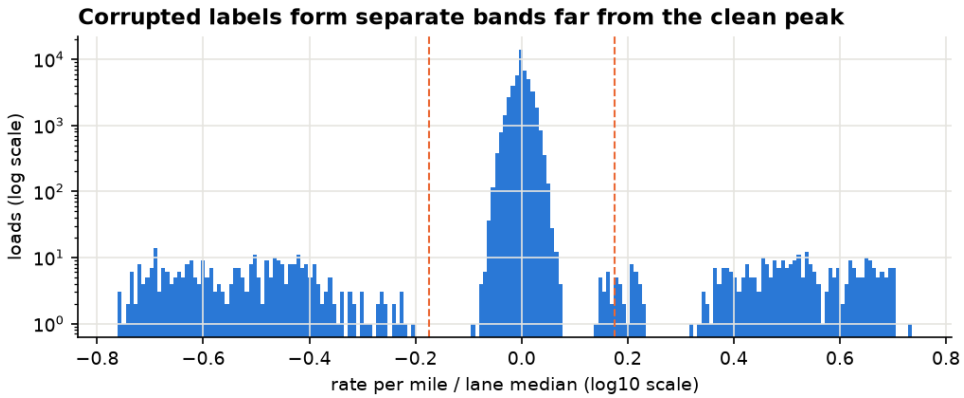


Figure 1: Rate per mile relative to the lane median. Dashed lines: the 0.67 and 1.5 screening bounds.

Treatment. Inside every training fold a robust two-pass ridge fit flags rows outside 0.67–1.5 times its prediction, and those rows are left out of training. The screen is learned, so it never sees assessment rows. For reporting we use a separate fixed flag (the lane-median ratio above). It marks the same kind of row for every model and is never used in fitting. Removing the screen costs $14.18 on the development folds (Section 5).

Negative weights. 292 training loads have negative weights whose sizes match normal weights. These are sign-entry errors, so we use the absolute value.

Weight cap. 1,204 loads sit exactly at 47,500 lb, a ceiling. A flag tells the model a weight is capped.

Missing values. 300 weights are missing; we fill them with 31,000 lb (about the median), and the tree stage sees them as missing. 374 market-index readings are missing; we fill each with that day's mean, because readings vary little within a day (standard deviation 0.025) compared with between days (0.167).

Why quote_signal is dropped. In training months, `quote_signal` either copies the rate per mile (correlation 0.88 to 0.99) or mirrors it (-0.82 to -0.94), depending on the month (Figure 2). In the scoring months that relationship is gone: its correlation with log distance is 0.01, against

a typical 0.81 in training. A model that used it would learn a leak that does not exist when predictions are made. It is never a model input, and a unit test checks that predictions do not change when it is replaced by noise. Load IDs and city names are not inputs either.

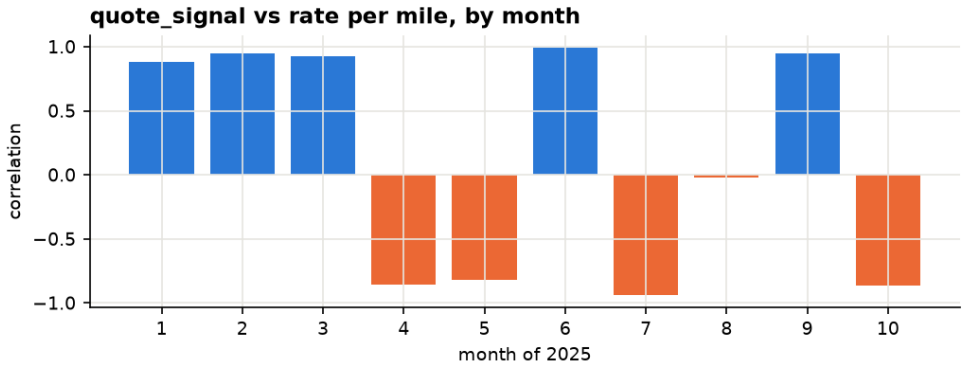


Figure 2: Monthly correlation of `quote_signal` with the rate per mile (clean training rows).

4 Validation design

Rolling-origin folds. A fold mimics the real situation once. We pick a cut-off date (the "origin"), train on every load up to it, and score the loads of the next 61 days. We use five origins: the last day of February, March, April, May and June 2025. Each fold's training window grows with time ("expanding window"). This is the standard way to evaluate forecasts [3]. A random split would mix future and past loads and reward features that only work by looking ahead.

Hidden cities. In every fold, 8 random cities (fixed seed: 100 plus the fold number) are removed from training. Their loads still appear in the scored period. This mimics the 12.06% of scoring loads that touch new cities.

Locked test. The last origin, 31 August, scores September and October. Nothing about the model was chosen on it. It was evaluated once, after the final configuration was frozen, by a script that refuses to run twice. One honest caveat: earlier analyses of this project had already looked at these months, so the test is not fully untouched. No untouched labelled period exists.

Metric. We report the mean absolute error (MAE) in dollars on loads with plausible labels. Loads touching hidden cities and other loads are re-weighted to the scoring mix (12.06% and 87.94%). We call this *wMAE*. We chose it for three reasons.

- It is in dollars, the unit of the business question.
- It is robust to the corrupted labels. On the locked test those 1.41% of loads produce 59% of all absolute error.
- It matches the median forecast that the log model produces.

The root mean squared error (RMSE) squares each error, so a handful of 3-to-5-times label errors dominate it. On the test period every model in this report scores an RMSE between $630 and $669, whatever its quality, so RMSE cannot rank models here. We still report it, together with MAPE (mean absolute percentage error) and bias.

Comparing two models. Differences come with a 95% interval from a paired block bootstrap. Both models are scored on the same loads; we resample whole 7-day blocks of days, because loads on nearby days share the market and are not independent [6].

5 Feature engineering

Each feature below has a plain definition, the reason it should help, and the result of removing it (or adding it) on the five development folds. The removals are made one at a time on the untuned ridge-plus-trees model (ladder step 5, $42.67), following the advice to test feature groups against out-of-sample error rather than importance scores [5]. Table 1 has every result with its paired interval.

Load description. Smooth curves (cubic B-splines) of log distance, weight and the four pickup and delivery coordinates, a weight-at-cap flag, and equipment dummies with Dry Van as the reference. Price per mile falls smoothly with distance and varies smoothly with location. Coordinates, unlike city names, also work for cities the model has never seen.

market_level_today. The logarithm of the day's mean market index. Its coefficient is an elasticity: the fitted model moves the rate by 0.154% for each 1% change in the day's index. Removing the market costs $34.11, the largest single loss.

market_deviation_this_load. The load's own log index minus the day's mean (elasticity 0.127).

days_since_start. A linear drift beyond the market, 0.62% per month in the final model. Removing it costs $13.34; without it every later window is under-forecast.

quarter_end_progress × equipment. Zero on the 1st and one on the last day of March, June, September and December; zero in other months. Prices ramp up through these months, most for Flatbed. By the last day the final model adds 2.3% for Dry Van, 5.0% for Reefer and 7.8% for Flatbed. Removing it costs $28.99: without it the March and June ramps leak into the drift term and the drift overshoots.

distance_x_market. (log distance − 6.8) × market_level_today: long hauls react a little more to the market. Removing it costs $0.28.

Weekday dummies (rejected). Adding them costs $3.00. The market index already carries the weekly price cycle.

Residual trees. The LightGBM stage sees distance, weight, coordinates, equipment and both market features. Hiding the market features from it costs $0.86.

Table 1: One change at a time against ladder step 5 (dev folds). Positive change = worse.

Change to the step-5 model	Dev wMAE ($)	Change ($)	Paired 95% CI ($)	Run
remove the market index	76.79	+34.11	[+24.65, +40.98]	33490709c76f
remove the drift term	56.02	+13.34	[+11.64, +14.95]	a1cecf3fa02f
remove the quarter-end ramp	71.66	+28.99	[+24.83, +32.45]	a42ce529d5aa
remove distance x market	42.95	+0.28	[+0.19, +0.33]	92d40fda9ff8
add weekday dummies	45.68	+3.00	[+2.09, +4.11]	e10ca665c3bd
keep corrupted labels	56.85	+14.18	[+13.08, +15.37]	2492292cd7e1
residual trees without market inputs	43.53	+0.86	[+0.62, +1.09]	0185e5309ed5

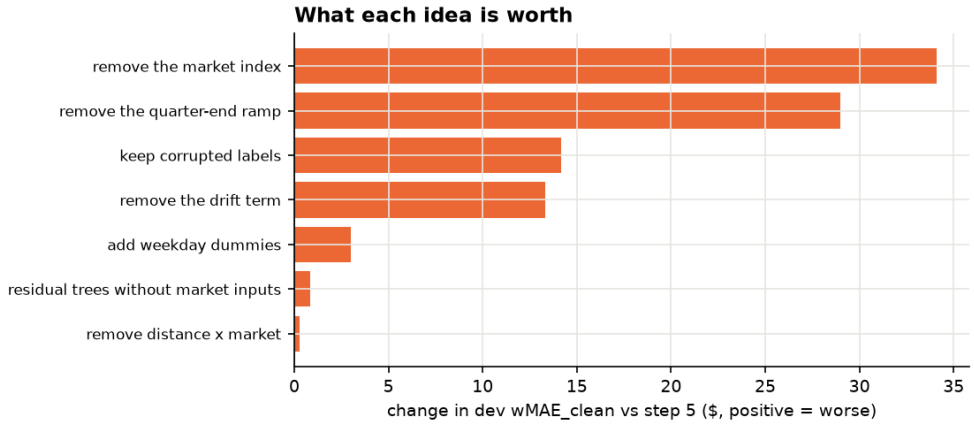


Figure 3: The same ablations as a chart.

6 Improvement journey

The final model came out of a sequence of experiments. Each step changed one idea, motivated by what the previous step got wrong. All six steps were re-run on the same five development folds for this report (Table 2, Figure 4).

1. **Equipment rate per mile × distance** ($157.55). The simplest sensible price. It knows nothing about place, weight or time.
2. **CatBoost on raw dollars** ($111.25). The original submission. It learns place and weight but fits dollars with a squared loss, so the corrupted labels pull it around, and its month features never saw a December.
3. **LightGBM on log rate with the label screen, market and calendar** ($63.77). Working on the log scale and dropping corrupted labels fix step 2's two worst problems. But trees cannot extend a trend or a market level beyond what they saw, so errors stay large when the level moves.
4. **Structural ridge** ($44.01). Time and market enter as straight lines on the log scale, so they extrapolate. This step removes most of the remaining error.
5. **Ridge plus residual trees** ($42.67). A small LightGBM learns the local patterns the ridge misses, such as a lane's geography interacting with distance and weight.
6. **Tuned and recency-weighted** ($42.30). Tuned spline knots and a larger, slower tree stage (Section 7). Recent loads weigh more, with a time constant of 122.6 days.

Table 2: The improvement ladder on the five development folds. "Change" is relative to the previous step.

Step	Model	Dev wMAE ($)	MAPE	Worst fold ($)	Change ($)
1	Equipment rate/mile x distance	157.55	8.56%	171.42	
2	CatBoost, raw dollars	111.25	5.74%	176.73	-46.30
3	LightGBM, log + screen + market	63.77	2.71%	75.96	-47.48
4	Structural ridge	44.01	1.84%	52.11	-19.76
5	Ridge + residual trees	42.67	1.79%	50.81	-1.34
6	Final (tuned, recency-weighted)	42.30	1.78%	50.62	-0.38

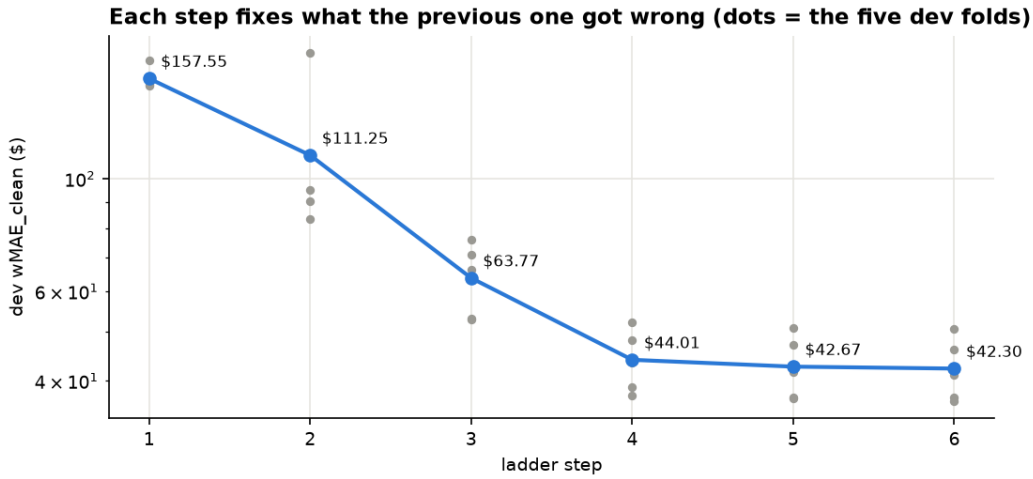


Figure 4: Development error by ladder step (log scale). Grey dots are the five folds.

Simple time-series baselines sit between steps 2 and 3. The best of them, a cross-section model with the last daily price level carried forward, scores $74.61 (Table 3).

Table 3: Baselines on the development folds.

Baseline	Dev wMAE ($)	MAPE	Run
Equipment rate/mile x distance	157.55	8.56%	59dc6251994c
Lane rate/mile x distance	144.84	5.93%	dd16e944cb36
Load description + naive level	74.61	3.10%	d9d6136ea962

Table 4: Paired 7-day block-bootstrap comparisons on the development folds (final minus other model; negative means the final model is better).

Comparison	Difference in clean MAE, 95% CI ($)	Folds final better
Final vs step 5 (untuned)	-0.36 [-0.45, -0.30]	5/5
Final vs step 4	-1.66 [-1.93, -1.41]	5/5
Final vs step 3	-22.12 [-26.52, -18.26]	5/5
Final vs step 2	-68.33 [-74.28, -62.52]	5/5
Final vs step 1	-114.47 [-116.85, -111.05]	5/5

7 Tuning

Search. Optuna [1] ran 60 trials of its tree-structured Parzen estimator (seed 42), each scored as the mean development wMAE over the five folds. The locked test was never used. The search covered the ridge penalty (0.001–10), spline knots (distance 5–12, coordinates 4–9, weight 4–7), the tree stage (50–600 trees, learning rate 0.01–0.15, 7–63 leaves, 20–300 loads per leaf, L2 penalty 0.001–30, 50–100% of columns per tree) and recency weighting (off, or a time constant of 45–720 days).

Result. The best trial scores $42.30; the untuned model scores $42.67. The best trial is the configuration frozen in `configs/final.yaml` (re-found: yes). Table 5 lists it. The tree stage's size

and learning rate matter most (Table 6).

Table 5: Tuned settings.

Setting	Tuned value
Ridge penalty alpha	0.265
Spline knots: distance / weight / coordinates	5 / 7 / 4
Recency time constant (days)	122.6
Residual trees	293
Learning rate	0.0215
Leaves per tree	62
Minimum loads per leaf	90
L2 penalty lambda	3.26
Share of columns per tree	0.81

Table 6: Most important settings.

Hyperparameter	Importance (fANOVA)
n estimators	0.37
learning rate	0.14
num leaves	0.12
min child samples	0.10
dist knots	0.09

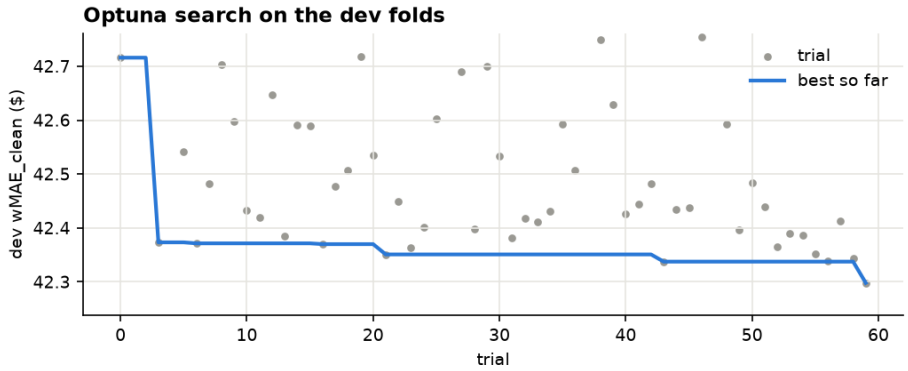


Figure 5: Trial scores and the best score so far.

Is the gain real? Tuning and scoring on the same folds flatters the tuned model. To check, we re-tuned on folds 1 to $k-1$ only and scored fold k (Table 7). The gains on these unseen folds, \$0.50 and \$0.48, are close to the in-sample gain of \$0.36.

Table 7: Forward-nested check: tune on earlier folds, score the next one.

Scored fold	Tuned on folds	Default ($)	Tuned ($)	Gain ($)
4	1–3	36.88	36.39	+0.50
5	1–4	47.00	46.53	+0.48

Honest note. On the locked test the untuned model was slightly better (Section 8). Tuned and default settings differ by about 1.1% on the test, and by similar amounts in the other direction on the development folds. They are practically equivalent. We kept the tuned model because the selection rule was fixed before the test was opened.

8 Final evaluation

Locked test. Table 8 shows every model on September and October 2025, scored once. The final model's wMAE is \$38.14 (1.56% of the rate). The original CatBoost submission scores \$64.31; its

error is $27.35 higher (95% interval 22.60 to 32.21). The untuned step-5 model scores $37.88: a difference of −0.42 dollars (interval −0.81 to −0.05), slightly in its favour.

Table 8: Locked test, September–October 2025. "vs final" is the other model's clean MAE minus the final model's, with a 95% paired block-bootstrap interval. Bias is the median log error.

Model	wMAE ($)	MAPE	All rows MAE ($)	RMSE ($)	Bias	vs final, 95% CI ($)
Final (tuned, recency-weighted)	38.14	1.56%	91.86	630.43	-0.58%	
Ridge + residual trees	37.88	1.55%	91.45	630.34	-0.52%	-0.42 [-0.81, -0.05]
Structural ridge	39.40	1.60%	92.88	630.65	-0.69%	+1.04 [+0.15, +2.08]
LightGBM, log + screen + market	48.94	2.08%	103.01	633.29	-1.27%	+11.30 [+8.01, +14.42]
CatBoost, raw dollars	64.31	3.74%	118.79	633.24	+0.69%	+27.35 [+22.60, +32.21]
Equipment rate/mile x distance	175.52	8.13%	226.67	669.09	-0.72%	+136.88 [+131.58, +143.86]
Lane rate/mile x distance	90.66	3.80%	143.74	662.70	-0.80%	+50.75 [+46.00, +56.29]
Load description + naive level	73.98	3.07%	126.82	639.30	-2.86%	+35.45 [+28.68, +44.51]

Including corrupted labels. On all rows the final model's MAE is $91.86. The difference from the clean figure is almost entirely the corrupted labels, which no model can predict.

Segments. Errors are similar across equipment types and for new versus known cities, and they grow with the forecast horizon (Table 9).

Table 9: Final model on the locked test by segment (clean rows).

Segment	Loads	MAE ($)	MAPE	Bias
Equipment: Dry Van	5275	33.77	1.51%	-0.58%
Equipment: Flatbed	1746	49.83	1.78%	-0.67%
Equipment: Reefer	2358	39.78	1.52%	-0.49%
Days ahead: 1-7	1078	29.81	1.34%	-0.14%
Days ahead: 8-14	1115	31.31	1.33%	+0.09%
Days ahead: 15-28	2125	34.46	1.42%	+0.07%
Days ahead: 29-42	2137	40.88	1.63%	-0.94%
Days ahead: 43-61	2924	44.92	1.78%	-1.17%
City: seen cities	6804	38.04	1.54%	-0.56%
City: unseen city	2575	38.88	1.62%	-0.62%

Residuals. Figure 6 shows the remaining error. From early October, actual prices sit about 1.1% above the forecast (median, October). Daily errors are strongly autocorrelated, which means a level shift that no available input explains.

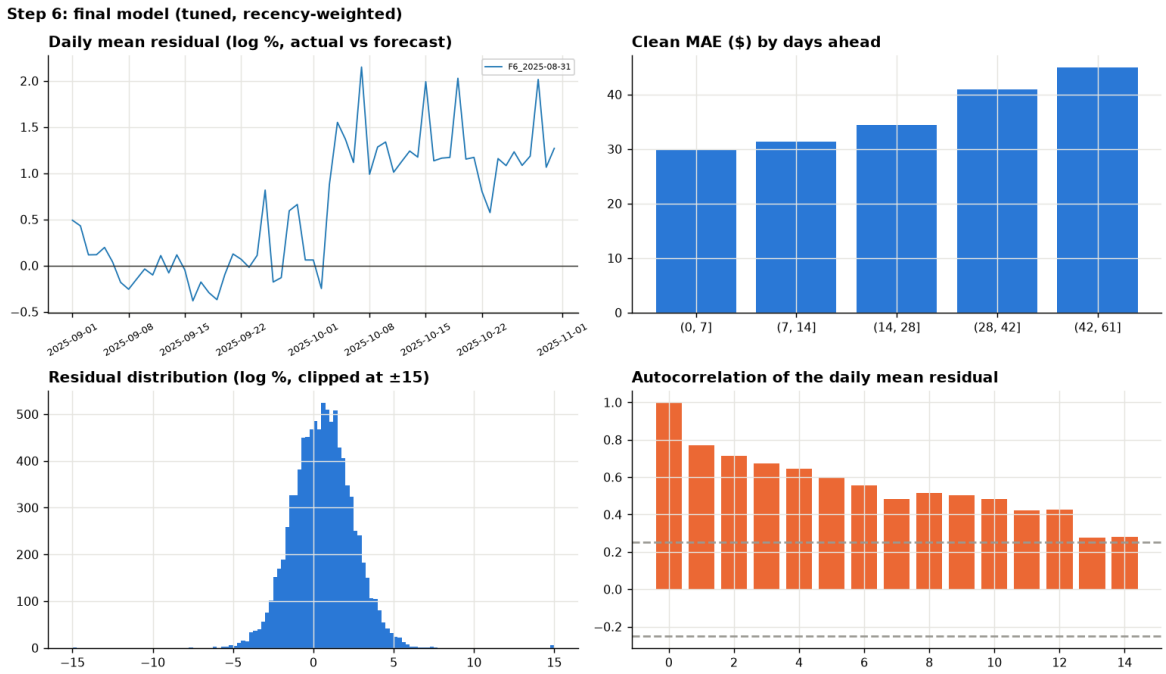


Figure 6: Final model on the locked test: daily mean residual, error by horizon, residual distribution and the autocorrelation of daily residuals.

Prediction intervals. We take the 10th/90th and 2.5th/97.5th percentiles of the out-of-sample log errors on the development folds, per equipment type. On the locked test the 80% interval covers 84.7% and the 95% interval 96.9% of clean loads (83.5% and 95.5% of all loads). The intervals are slightly conservative.

9 December forecast

The chart rows have no market index and no coordinates. We look the coordinates up from the city list, and take each day's market index as the mean over that day's loads in `validation.csv`.

The forecast runs from $829.08 to $877.62 and ends at $877.62 on December 31 (Figure 7). The weekly wave follows the market index. The slow rise is the drift plus the quarter-end ramp.

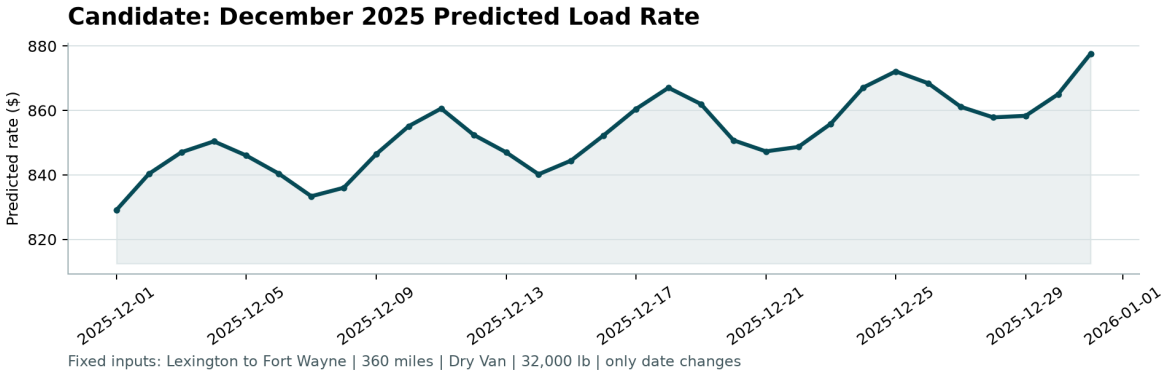


Figure 7: The December chart produced by the assessment's own `score.py`.

How much rests on the quarter-end ramp? December has never been observed. The ramp was seen in March, June and September, and the model assumes it recurs. Without it, December

31 would be $857.83 instead of $877.62. If the December market index were unknown and held at its October mean, the line would be a smooth $838.79 to $863.82 (Figure 8).

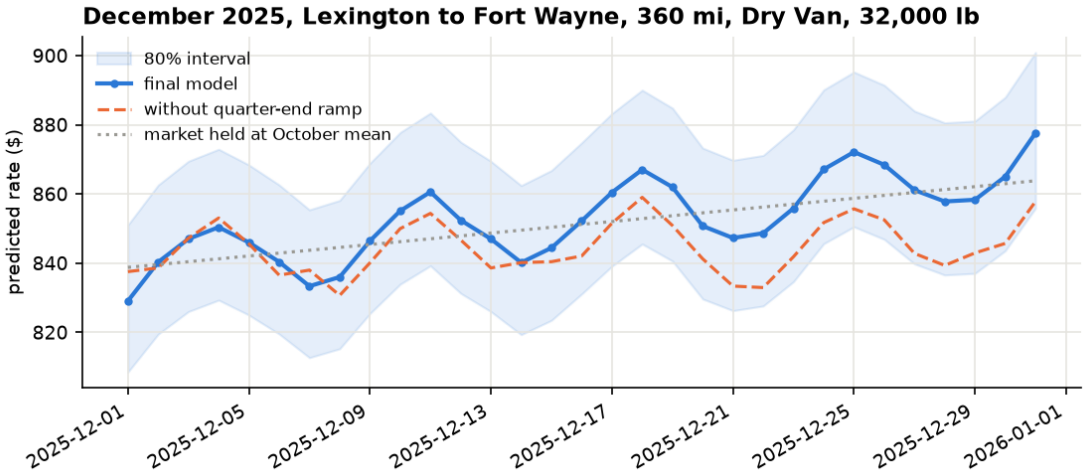


Figure 8: December forecast with its 80% interval, without the quarter-end ramp, and with the market held at its October mean.

10 Limitations

- **Test contamination.** Earlier analyses had already looked at September and October, and the quarter-end feature was confirmed there. The locked-test numbers are therefore mildly optimistic.
- **December is an extrapolation.** The labels contain no December. The forecast assumes the quarter-end ramp recurs, and holiday effects cannot be learned.
- **Level shifts.** The model cannot anticipate a sudden change in the price level that the market index does not show, like the step in early October. In October the actual level sat 1.1% above the forecast.
- **Market index dependence.** The market index is the most important input. Without the target day's index, accuracy falls towards the no-market models ($34.11 worse on the development folds).
- **Label noise in scoring.** If the hidden labels contain the same 1.41% of errors, the all-row error will stay near $91.86 whatever the model.
- **Synthetic-looking data.** City coordinates are shifted and the data look simulated. Statements about freight markets describe this dataset only.
- **Out-of-range geography.** Laredo lies 2.9 degrees of latitude south of every training city, so its prices extend the coordinate curves beyond the data.

11 Reproducibility

Everything runs in Docker. You need Docker with Compose v2, `make`, and the four CSV files in `data/`.

```
cp .env.example .env        # choose a database password
make all                    # about 15 minutes on 2 CPU cores
```

`make all` runs, in order: `up` (PostgreSQL 16 and an MLflow 3.5 server with persistent volumes), `data-check` (schema and SHA-256 of the inputs), `experiments` (baselines, the six ladder steps and seven ablations), `tune` (60 Optuna trials and the nested check), `test-lock` (the one-shot locked test), `train` (final fit, export to `model/`, registration as `freight-rate-model@champion`), `predict` (submission files checked by `score.py`), `report` (this PDF), `test` and `lint`.

Every run compares its result with the value logged by the earlier project and stops if they differ by more than five cents. Seeds are fixed: LightGBM 42, Optuna 42, bootstrap 7, hidden cities 100 plus the fold number. Each MLflow run records the dataset hashes, git commit, configuration file, parameters, fold manifests, per-fold and mean metrics, out-of-fold predictions and a residual plot. The MLflow interface is at `http://localhost:5050`.

The champion training run is `f51bb0ec64524e3eb1d170f3ee1765a4` and the locked-test run of the final model is `95fb98149167470195 28b05e5b28c031`. Fitting takes 1.8 seconds on all 47,323 screened loads, and inference takes about 46 microseconds per load.

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019.

[2] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

[3] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3rd edition, 2021.

[4] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, 2017.

[5] Max Kuhn and Kjell Johnson. *Feature Engineering and Selection: A Practical Approach for Predictive Models*. CRC Press, 2019.

[6] Hans R. Künsch. The jackknife and the bootstrap for general stationary observations. *The Annals of Statistics*, 17(3):1217–1241, 1989.

A Final model parameters

Table 10: Resolved parameters of the registered champion (from `model/final_params.json`).

Parameter	Value
ridge.alpha	0.2649564339357501
ridge.groups	["load", "market", "trend", "quarter_end", "distance_x_market"]
ridge.recency_tau_days	122.61806047369296
ridge.spline_knots	"distance_knots": 5, "weight_knots": 7, "coordinate_knots": 4, "coordinate_extrapolation": "linear"
ridge.rows_used_after_label_screen	47323
ridge.rows_available	48000
residual_lightgbm.n_estimators	293
residual_lightgbm.learning_rate	0.02145244449350786
residual_lightgbm.num_leaves	62
residual_lightgbm.min_child_samples	90
residual_lightgbm.subsample	0.8
residual_lightgbm.subsample_freq	1
residual_lightgbm.colsample_bytree	0.8148594592943516
residual_lightgbm.verbose	-1
residual_lightgbm.random_state	42
residual_lightgbm.n_jobs	1
residual_lightgbm.reg_lambda	3.261559198325916

Table 11: Ridge coefficients of the non-spline terms (log-rate change per unit; the trend is per 100 days).

Feature	Coefficient
weight_at_cap	-0.0016
equipment_flatbed	+0.0729
equipment_reefer	+0.1175
market_level_today	+0.1538
market_deviation_this_load	+0.1273
hundreds_of_days_since_start	+0.0208
quarter_end_progress_x_dry_van	+0.0232
quarter_end_progress_x_flatbed	+0.0782
quarter_end_progress_x_reefer	+0.0495
distance_x_market	+0.0148
intercept	+7.3542

B MLflow run inventory

Table 12: Every top-level run behind this report (Optuna trials are nested under `optuna-search`).

Experiment	Run name	Run ID
01-baselines	Baseline: equipment rate per mile x distance	59dc6251994c4dd29f6e3ac87ede80f8
01-baselines	Baseline: lane rate per mile x distance	dd16e944cb3642aca46db4ce02a466ee
01-baselines	Baseline: load description + naive daily price level	d9d6136ea9624145ad53740f73b1a270
02-improvement-ladder	Step 1: equipment rate per mile x distance	f03a9cd49d154b9086b769b786dc29a2
02-improvement-ladder	Step 2: CatBoost on raw dollars (original submission)	35078756fc8f4814b0e74d8afc9e6507
02-improvement-ladder	Step 3: LightGBM, log target + label screen + market + calendar	4b894229640948979217ce982a4828fb
02-improvement-ladder	Step 4: structural ridge (splines, coordinates, market, trend, quarter-end)	e10eb35b0f554f30bf8f4bc245b06f25
02-improvement-ladder	Step 5: ridge + LightGBM on residuals (default settings)	bb1c3f0a4fbc45e6861f3e0ff29a3a27
02-improvement-ladder	Step 6: final model (tuned, recency-weighted)	30477774dc994856b2eae8bfd1ba7ca3
03-ablations	Ablation: remove the market index	33490709c76f463e9a0db586d291ec1b
03-ablations	Ablation: remove the drift term	a1cecf3fa02f46ac9a168b782a53bd84
03-ablations	Ablation: remove the quarter-end ramp	a42ce529d5aa4460995ace8368c05542
03-ablations	Ablation: remove distance x market	92d40fda9ff84b80809f5a463a75eb0e
03-ablations	Ablation: add weekday dummies	e10ca665c3bd4d9094f3620baf7dc99c
03-ablations	Ablation: keep corrupted labels	2492292cd7e14169bd75062039a0167b
03-ablations	Ablation: residual trees without market inputs	0185e5309ed5476ea01f60bf07b1a1bb
04-tuning	optuna-search	716ee18304584bbf8962b9ea4859e9f7
04-tuning	nested-forward-check	81b4968953e34dc9979a6f517dcf0b06
04-tuning	optuna-search	dee389aba0bd45deb18cac72152ba96e
04-tuning	nested-forward-check	e5ab30f226be41d8a68302515d350fc1
05-locked-test	Step 6: final model (tuned, recency-weighted)	95fb9814916747019528b05e5b28c031
05-locked-test	Step 5: ridge + LightGBM on residuals (default settings)	753243be347f4710b85ff8f1a8f61fc8
05-locked-test	Step 4: structural ridge (splines, coordinates, market, trend, quarter-end)	ce3e1eb6a8514b28985165157194cc42
05-locked-test	Step 3: LightGBM, log target + label screen + market + calendar	339e36665d6c4bb094c4652058e601fe
05-locked-test	Step 2: CatBoost on raw dollars (original submission)	08450efd7cce40b39efc2687ff9aabe5
05-locked-test	Step 1: equipment rate per mile x distance	f7e1a7bceb94498c8a421cca0296961c
05-locked-test	Baseline: lane rate per mile x distance	64409dcbe41d48278c508e5c01f2e5e4
05-locked-test	Baseline: load description + naive daily price level	63e20dd3b45641729679c755bbfd6621
06-final	champion: ridge + residual trees (tuned)	f51bb0ec64524e3eb1d170f3ee1765a4